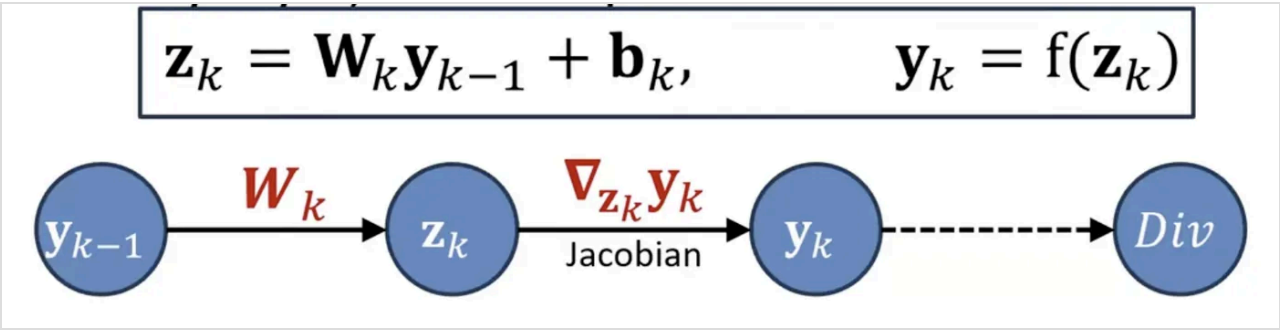
Lec6. Training Part IV

- Gradient Descent
  Training the network

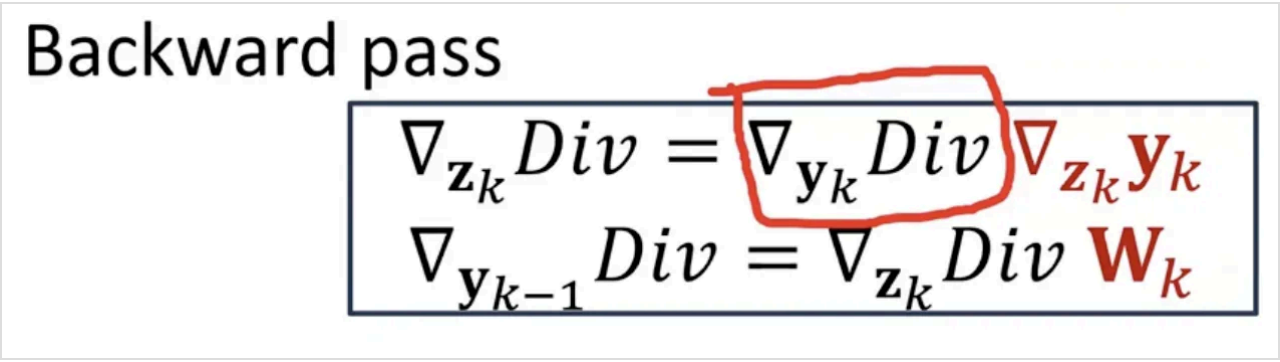
Vector derivatives

- at any layer, **forward pass**



- affine항으로 z값을 구하고 → activation function f(z)를 구하면 됨.
  이때 화살표 위에는, 미분값을 표시하게 되는데
  affine항의 경우, linear하게 W_k가 되고,
  activation은 **jacobian**이 됨.

Backward pass



- loss(div)를 구할 때에는, 이렇게 화살표 위의 미분값들을 곱하는 방식(chain rule)을 통하게 됨.

$$\nabla_{\mathbf{b}_k} Div = \nabla_{\mathbf{z}_k} Div, \qquad \nabla_{\mathbf{W}_k} Div = \mathbf{y}_{k-1} \nabla_{\mathbf{z}_k} Div$$

- input에 대한 미분값(학습을 위해)구할 때에는,
  위 방식을 통해 각 node에 저장된 미분값에서, w, b에 대한 미분을 하면 됨.

Neural network training algorithm

Initialize all weights and biases $(\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \ldots, \mathbf{W}_N, \mathbf{b}_N)$

Do:

- $Loss = 0$
- For all k, initialize $\nabla_{\mathbf{W}_k} Loss = 0$, $\nabla_{\mathbf{b}_k} Loss = 0$
- For all $t = 1:T$ # Loop through training instances
  - Forward pass : Compute
    - Output $\boldsymbol{Y}(\boldsymbol{X}_t)$,
    - Divergence $\boldsymbol{Div}(\boldsymbol{Y}_t, \boldsymbol{d}_t)$
  - Backward pass: For all k compute:
    - $\nabla_{\mathbf{W}_k} \boldsymbol{Div}(\boldsymbol{Y}_t, \boldsymbol{d}_t), \nabla_{\mathbf{b}_k} \boldsymbol{Div}(\boldsymbol{Y}_t, \boldsymbol{d}_t)$
    - $\nabla_{\mathbf{W}_k} Loss \mathrel{+}= \nabla_{\mathbf{W}_k} \boldsymbol{Div}(\boldsymbol{Y}_t, \boldsymbol{d}_t)$; $\nabla_{\mathbf{b}_k} Loss \mathrel{+}= \nabla_{\mathbf{b}_k} \boldsymbol{Div}(\boldsymbol{Y}_t, \boldsymbol{d}_t)$

  Computing gradient (uses backprop)

- For all k, update:

$$\mathbf{W}_k = \mathbf{W}_k - \frac{\eta}{T} \left(\nabla_{\mathbf{W}_k} Loss\right)^T; \qquad \mathbf{b}_k = \mathbf{b}_k - \frac{\eta}{T} \left(\nabla_{\mathbf{W}_k} Loss\right)^T$$

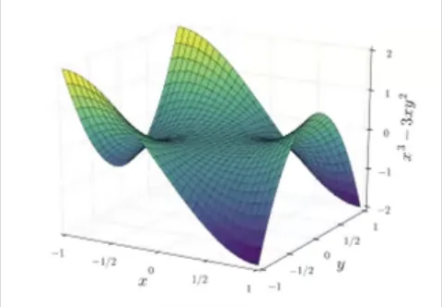
Gradient descent

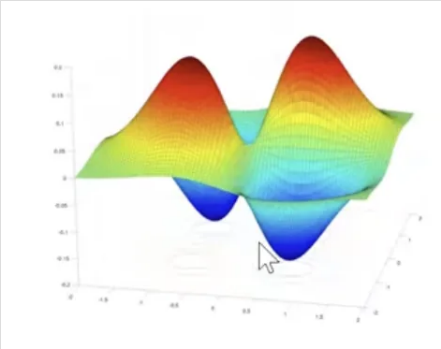
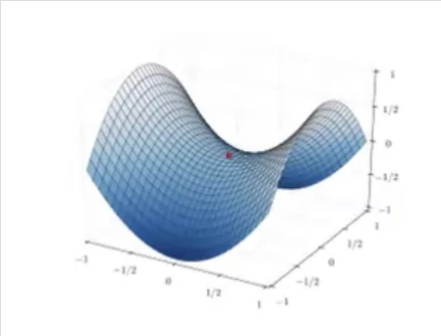
Until $Loss$ has converged

Is backprop always right?

- → loss function의 최소값을 찾는 게 항상, 최적의 분류를 만들지는 않음.

Loss Surface

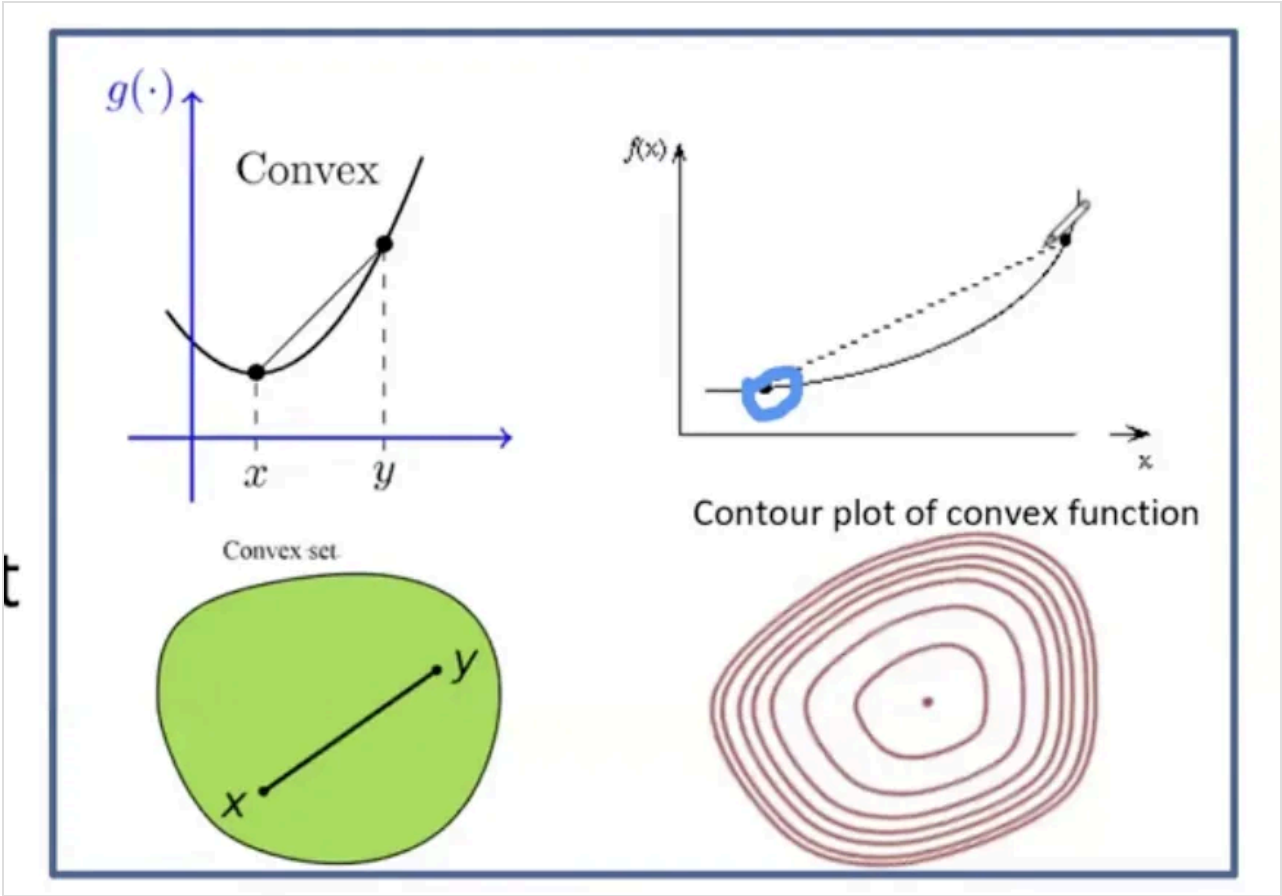


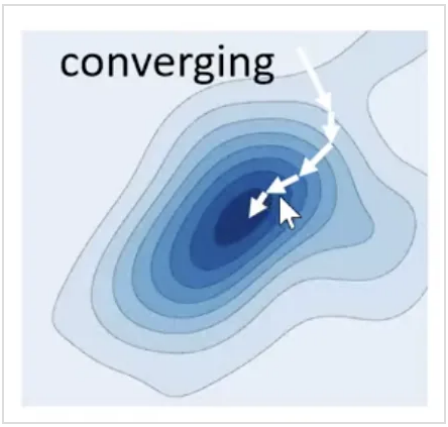
- popular hypothesis
  - in large networks
    → **saddle points** are far more common than local minima

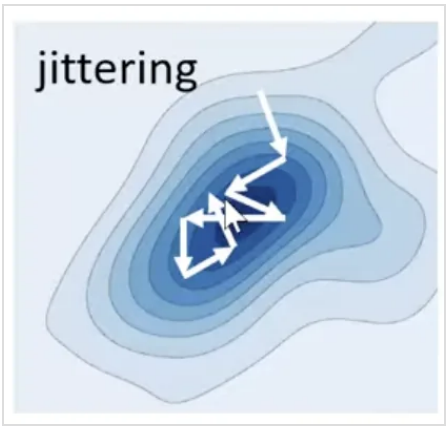
Convex optimization

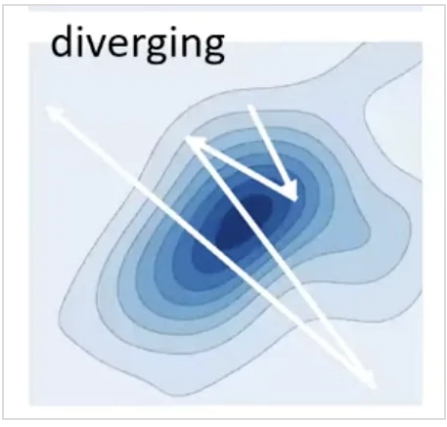
- → 함수들이 convex하다고 가정하고 문제들을 푸는 것임.



- convex할수록 최소화 하기 쉬움.







- convex할 때, gradient descent를 하면.
- converging할 수도, iterating할 수도, diverging할 수도 있음

Convergence for quadratic surfaces

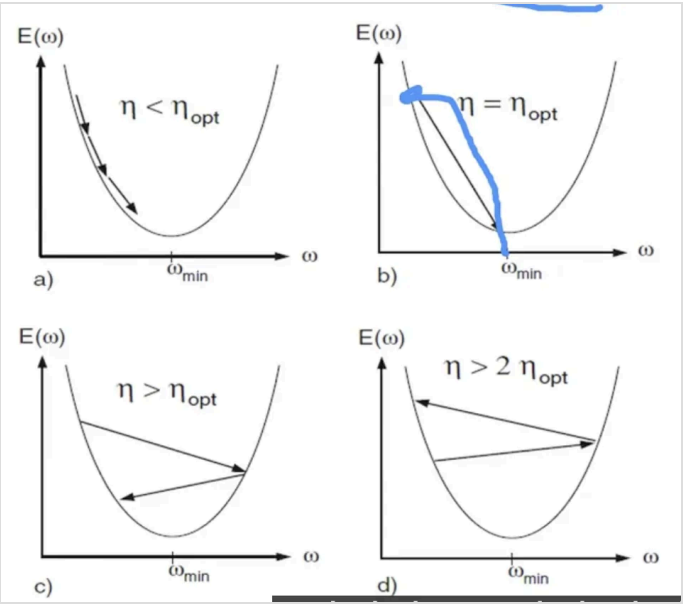
- $MinimizeE = \frac{1}{2}aw^2 + bw + c$

- → 아래와 같이 학습하면 됨.

$$w^{(k+1)} = w^{(k)} - \eta \frac{dE(w^{(k)})}{dw}$$

- 최적의 **step size**는?
- 테일러 전개에 의하면…
- $f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2} f''(x_0)(x - x_0)^2 + \dots$
- $W^{(k)}$를 기준으로 전개하면? → 3차 항은 없어짐.
  ⇒ 그래서 2차 근사가 될 거임.
- $E = E(w_k) + E'(w_k)(w - w_k) + \frac{1}{2} E''(w - w_k)^2$
- $E' = E'(w_k) + E''(w - w_k)$
  ⇒ 이게 0이라고 하면(최소인 지점)
- $(E'')^{-1} E' + (w - w_k) = 0$
  ⇒ $w^* = w_k - (E'')^{-1} E'$
- ⇒ $\eta = (E'')^{-1}$ 일 때 → 1번만에 최적점으로 이동할 수 있음.
  ⇒ **optimal step size**

With non-optimal step size



For $\eta < \eta_{opt}$ the algorithm will converge monotonically

For $2\eta_{opt} > \eta > \eta_{opt}$ we have oscillating convergence

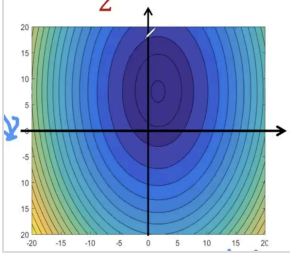
For $\eta > 2\eta_{opt}$ we get divergence

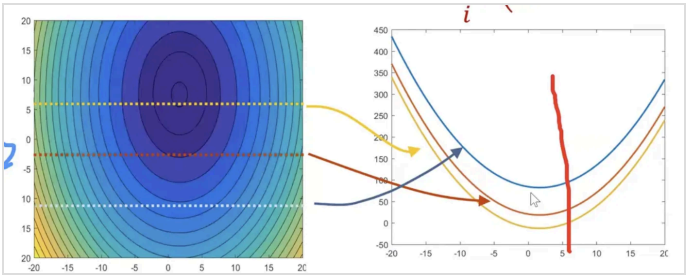
- 여러번 반복해서 최적점을 찾아가게 됨.

---

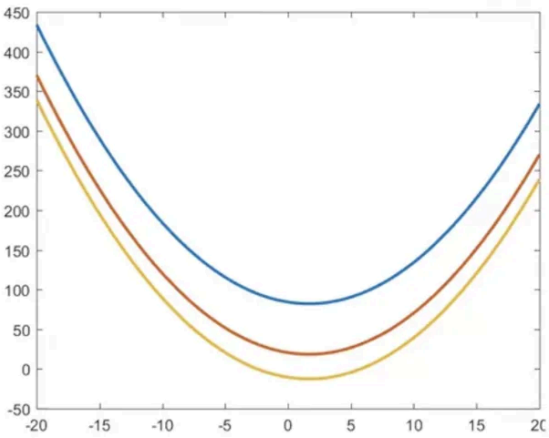
function of multivariate inputs

$$E = \frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w} + \mathbf{w}^T\mathbf{b} + c = \sum_i \left(\frac{1}{2}a_{ii}w_i^2 + b_i w_i\right) + c$$



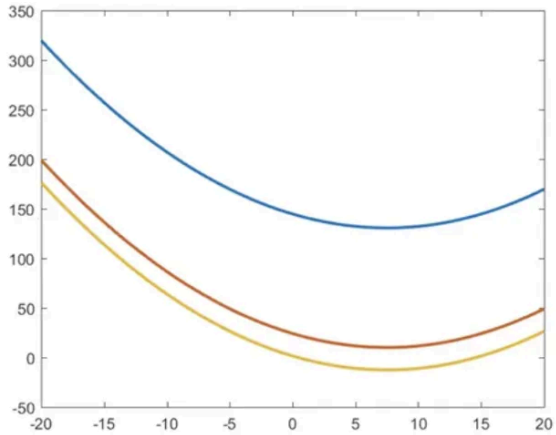


- w_2를 고정하면,
- w_1에 대한 그냥 함수로 보이게 됨.
- ⇒ 각자 independent가 있음.



$$E = \frac{1}{2}a_{11}w_1^2 + b_1 w_1 + c + C(\neg w_1)$$

$$\eta_{1,opt} = a_{11}^{-1}$$



$$E = \frac{1}{2}a_{22}w_2^2 + b_2 w_2 + c + C(\neg w_2)$$

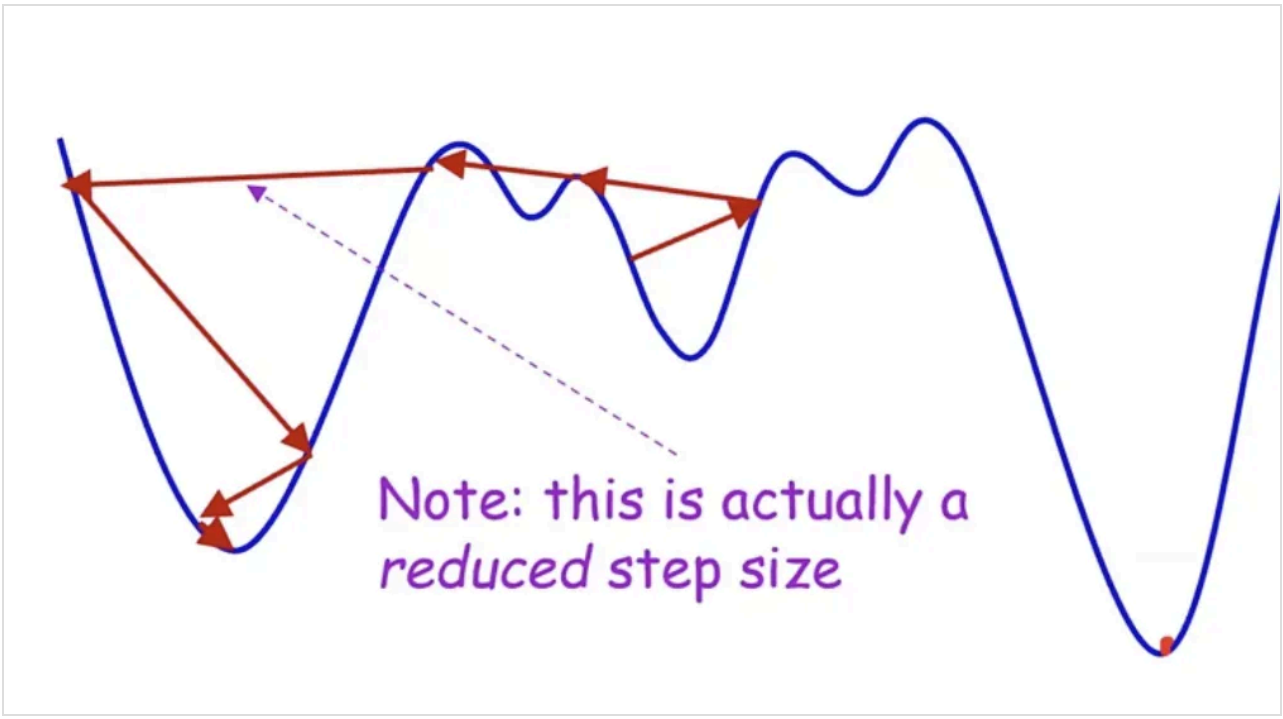
$$\eta_{2,opt} = a_{22}^{-1}$$

- ⇒ 서로 다른 **optimal step size**를 가지게 됨.
- 그럼 **multivariate**에서는 어떤 step size를 써야할까?
  divergence 하지 않게 충분히 작은 **step size**를 써야 함.

$$\eta < 2 \min_i \eta_{i,opt}$$

- → 충분히 커서 느리지 않으면서, divergence않을만큼 작은 step size를 찾아야 함.

Decaying learning rate



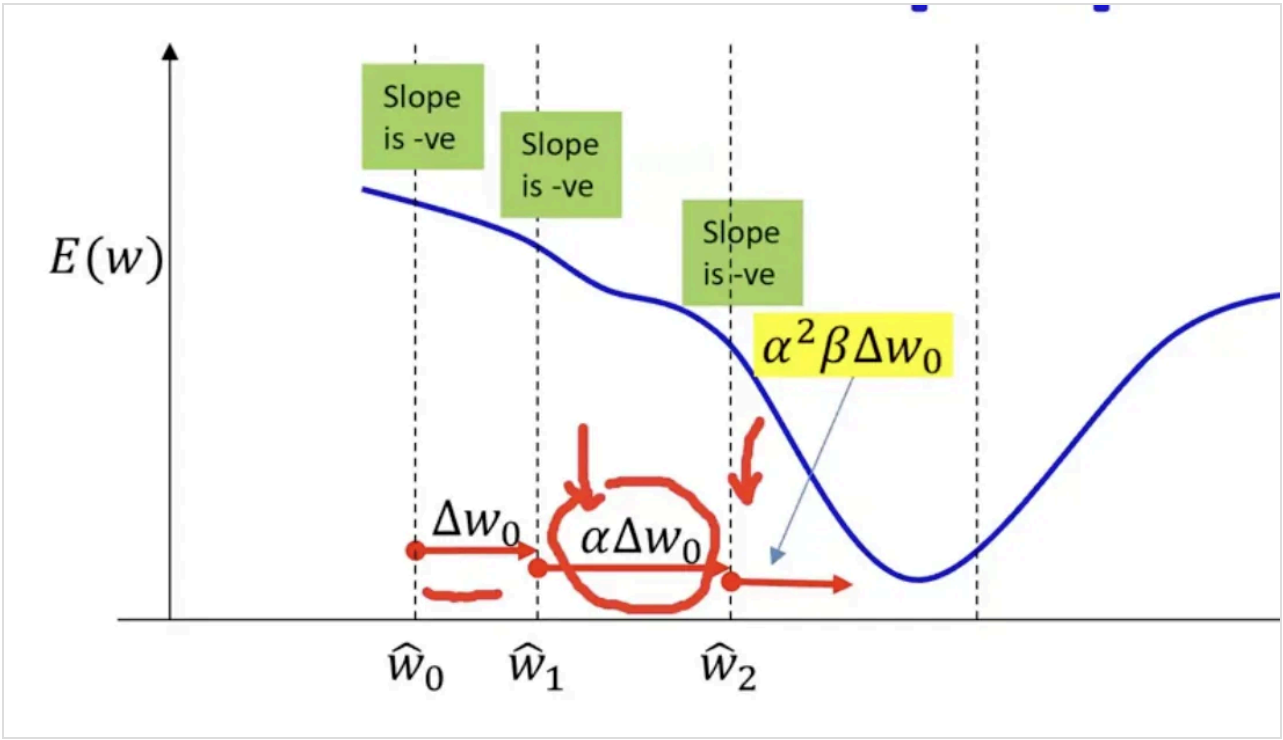
- Step size를 줄이면서 학습하면 됨.

- Linear decay: $\eta_k = \frac{\eta_0}{k+1}$
- Quadratic decay: $\eta_k = \frac{\eta_0}{(k+1)^2}$
- Exponential decay: $\eta_k = \eta_0 e^{-\beta k}$, where $\beta > 0$

Derivative-inspired algorithms

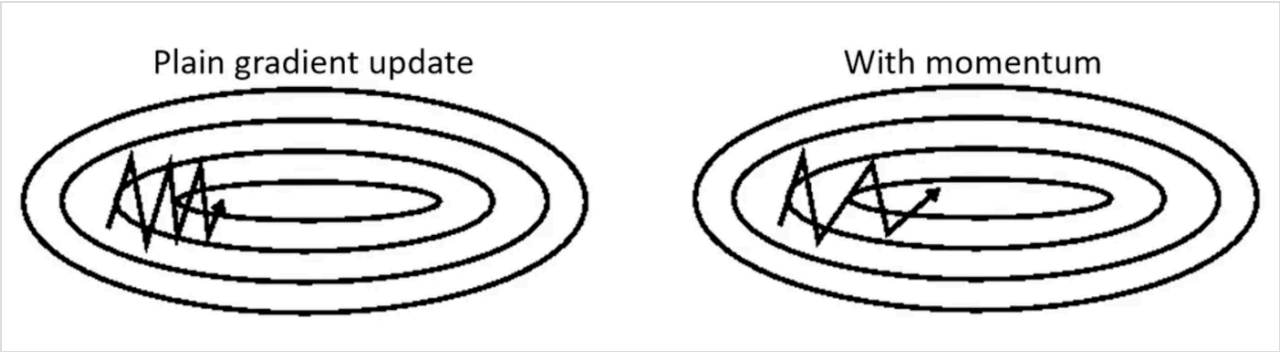
Rprop

- each time,
  방향이 전과 같으면 → lr을 늘려서(α를 곱해서) 가던 방향으로 감.
  - 근데 그러다가 부호가 바뀌면?
    돌아와서, lr을 줄인 뒤(β를 곱해서) 다시 감.



- **Step size가 충분히 커**져서 local minima를 넘어서,
  더 깊은 곳으로 갈 수 있음.
- **Quick prop**
  - Second derivatives를 추측해서 lr에 영향을 주는 방식
  - → 다음 수업에서 다룸

momentum



- derivates 값들의 **running average**을 이용함.
- $x_1, x_2, \ldots$이 있으면
  $\rightarrow \bar{x}_k = \alpha \bar{x}_{k-1} + x_k$

running average는

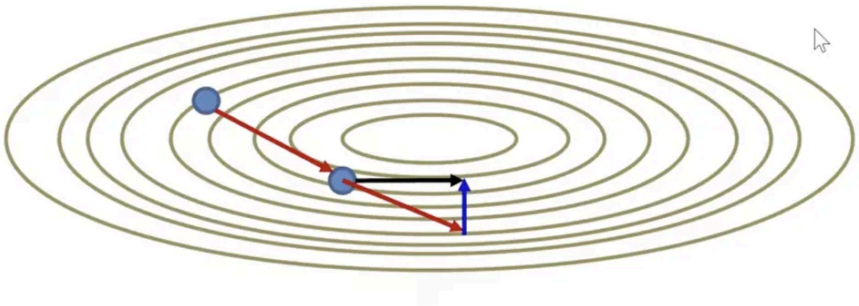
- x_k를 구할 때 이전까지의 평균을 축소한 후 평균을 구하는 것임.

$$gradientstep = -\eta \nabla_W Loss\left(W^{(k-1)}\right)$$

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} + gradientstep$$

$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

Nestorov's Accelerated Gradient



- Nestorov's method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W Loss\left(W^{(k-1)} + \beta \Delta W^{(k-1)}\right)^T$$

$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$